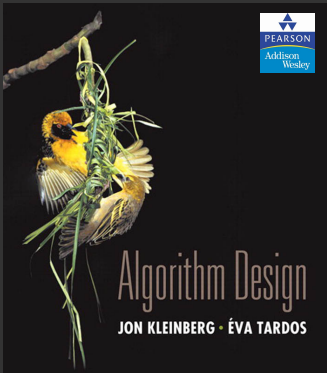




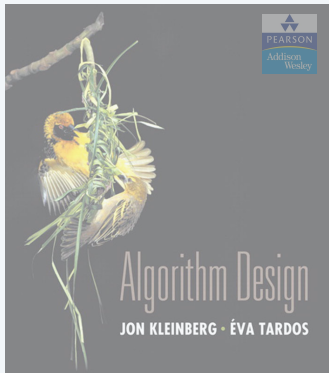
ĐƯỜNG ĐI NGẮN NHẤT

- ▶ *Đường đi ngắn nhất và thuật toán Dijkstra*
- ▶ *Đường đi ngắn nhất cho đồ thị DAG*
- ▶ *Thuật toán Bellman-Ford*
- ▶ *Thuật toán Floyd–Warshall*
- ▶ *Chu trình có trọng số âm*

Lecture slides by Kevin Wayne

Copyright © 2005 Pearson–Addison Wesley

<http://www.cs.princeton.edu/~wayne/kleinberg-tardos>



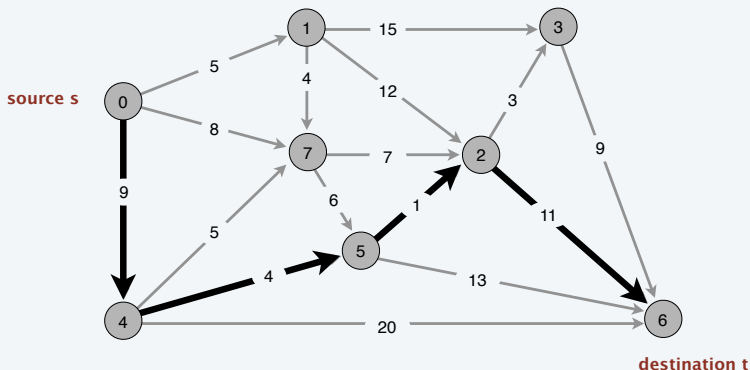
SECTION 4.4

ĐƯỜNG ĐI NGẮN NHẤT

- ▶ *Đường đi ngắn nhất và thuật toán Dijkstra*
- ▶ *Đường đi ngắn nhất cho đồ thị DAG*
- ▶ *Thuật toán Bellman-Ford*
- ▶ *Thuật toán Floyd-Warshall*
- ▶ *Chu trình có trọng số âm*

Bài toán đường đi ngắn nhất đơn cặp (single-pair SPP)

Bài toán. Cho một đồ thị (có hướng) $G = (V, E)$, độ dài các cạnh $\ell_e \geq 0$, nguồn $s \in V$ và đích $t \in V$, hãy tìm đường đi ngắn nhất từ s đến t .

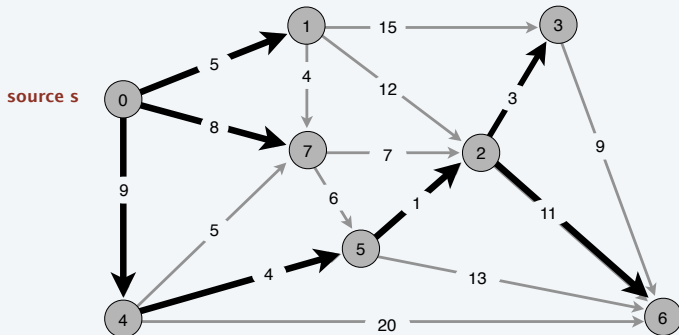


length of path = $9 + 4 + 1 + 11 = 25$

Bài toán đường đi ngắn nhất đơn cặp (single-pair SPP)

Bài toán. Cho một đồ thị (có hướng) $G = (V, E)$, độ dài các cạnh $\ell_e \geq 0$, nguồn $s \in V$ và đích $t \in V$, hãy tìm đường đi ngắn nhất từ s đến t .

Giả thiết. Tồn tại đường đi từ s đến mỗi đỉnh.



shortest-paths tree



**Suppose that you change the weight of every edge of G as follows.
For which is every shortest path in G a shortest path in G' ?**

- A.** Add 17.
- B.** Multiply by 17.
- C.** Either A or B.
- D.** Neither A nor B.



Which variant in car GPS?

- A. Single source: from one node s to every other node.
- B. Single sink: from every node to one node t .
- C. Source–sink: from one node s to another node t .
- D. All pairs: between all pairs of nodes.



Shortest path applications

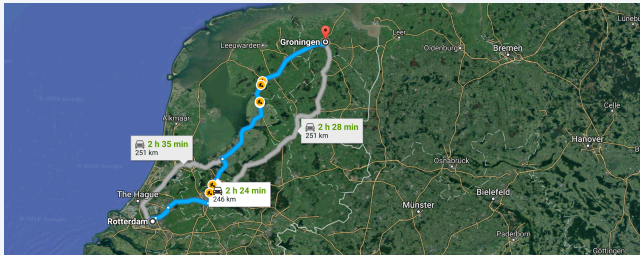
- PERT/CPM.
- Map routing.
- Seam carving.
- Robot navigation.
- Texture mapping.
- Typesetting in LaTeX.
- Urban traffic planning.
- Telemarketer operator scheduling.
- Routing of telecommunications messages.
- Network routing protocols (OSPF, BGP, RIP).
- Optimal truck routing through given traffic congestion pattern.



Network Flows: Theory, Algorithms, and Applications,
by Ahuja, Magnanti, and Orlin, Prentice Hall, 1993.

Edsger Dijkstra

“What’s the shortest way to travel from Rotterdam to Groningen? It is the algorithm for the shortest path, which I designed in about 20 minutes. One morning I was shopping in Amsterdam with my young fiancée, and tired, we sat down on the café terrace to drink a cup of coffee and I was just thinking about whether I could do this, and I then designed the algorithm for the shortest path.” — Edsger Dijkstra



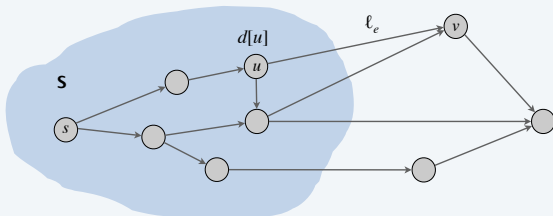
Thuật toán Dijkstra (cho bài toán đường đi ngắn nhất đơn cặp)

Cách tiếp cận tham lam (greedy). Duy trì một tập S các đỉnh đã được khám phá, tức là thuật toán đã tính được $d[u] = \text{length độ dài đường đi ngắn nhất } s \rightarrow u$ cho chúng.

- Khởi tạo $S \leftarrow \{s\}$, $d[s] \leftarrow 0$.
- Repeatedly choose unexplored node $v \notin S$ which minimizes

$$\pi(v) = \min_{e=(u,v) : u \in S} d[u] + \ell_e$$

the length of a shortest path from s to some node u in explored part S , followed by a single edge $e = (u, v)$



Dijkstra's algorithm (for single-source shortest paths problem)

Greedy approach. Maintain a set of explored nodes S for which algorithm has determined $d[u]$ = length of a shortest $s \rightarrow u$ path.



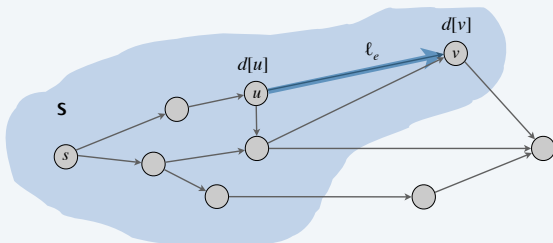
- Initialize $S \leftarrow \{s\}$, $d[s] \leftarrow 0$.
- Repeatedly choose unexplored node $v \notin S$ which minimizes

$$\pi(v) = \min_{e=(u,v): u \in S} d[u] + \ell_e$$

the length of a shortest path from s to some node u in explored part S , followed by a single edge $e = (u, v)$

add v to S , and set $d[v] \leftarrow \pi(v)$.

- To recover path, set $pred[v] \leftarrow e$ that achieves min.



Dijkstra's algorithm: proof of correctness

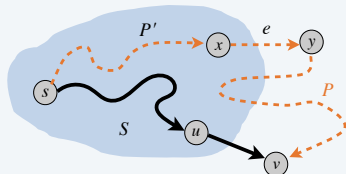
Invariant. For each node $u \in S$: $d[u]$ = length of a shortest $s \rightarrow u$ path.

Pf. [by induction on $|S|$]

Base case: $|S| = 1$ is easy since $S = \{ s \}$ and $d[s] = 0$.

Inductive hypothesis: Assume true for $|S| \geq 1$.

- Let v be next node added to S , and let (u, v) be the final edge.
- A shortest $s \rightarrow u$ path plus (u, v) is an $s \rightarrow v$ path of length $\pi(v)$.
- Consider **any** other $s \rightarrow v$ path P . We show that it is no shorter than $\pi(v)$.
- Let $e = (x, y)$ be the first edge in P that leaves S , and let P' be the subpath from s to x .
- The length of P is already $\geq \pi(v)$ as soon as it reaches y :



$$\ell(P) \geq \ell(P') + \ell_e \geq d[x] + \ell_e \geq \pi(y) \geq \pi(v) \quad \blacksquare$$

↑
non-negative
lengths

↑
inductive
hypothesis

↑
definition
of $\pi(y)$

↑
Dijkstra chose v
instead of y

Dijkstra's algorithm: efficient implementation

Critical optimization 1. For each unexplored node $v \notin S$:
explicitly maintain $\pi[v]$ instead of computing directly from definition



$$\pi(v) = \min_{e=(u,v) : u \in S} d[u] + \ell_e$$

- For each $v \notin S$: $\pi(v)$ can only decrease (because set S increases).
- More specifically, suppose u is added to S and there is an edge $e = (u, v)$ leaving u . Then, it suffices to update:

$$\pi[v] \leftarrow \min \{ \pi[v], \pi[u] + \ell_e \}$$

 recall: for each $u \in S$,
 $\pi[u] = d[u] = \text{length of shortest } s \rightarrow u \text{ path}$

Critical optimization 2. Use a min-oriented **priority queue** (PQ)
to choose an unexplored node that minimizes $\pi[v]$.

Dijkstra's algorithm: efficient implementation

Implementation.

- Algorithm maintains $\pi[v]$ for each node v .
- Priority queue stores unexplored nodes, using $\pi[\cdot]$ as priorities.
- Once u is deleted from the PQ, $\pi[u]$ = length of a shortest $s \rightarrow u$ path.

DIJKSTRA (V, E, ℓ, s)

FOREACH $v \neq s$: $\pi[v] \leftarrow \infty, pred[v] \leftarrow null; \pi[s] \leftarrow 0$.

Create an empty priority queue pq .

FOREACH $v \in V$: INSERT($pq, v, \pi[v]$).

WHILE (IS-NOT-EMPTY(pq))

$u \leftarrow$ DEL-MIN(pq).

 FOREACH edge $e = (u, v) \in E$ leaving u :

 IF ($\pi[v] > \pi[u] + \ell_e$)

 DECREASE-KEY($pq, v, \pi[u] + \ell_e$).

$\pi[v] \leftarrow \pi[u] + \ell_e; pred[v] \leftarrow e$.

Dijkstra's algorithm: which priority queue?

Performance. Depends on PQ: n INSERT, n DELETE-MIN, $\leq m$ DECREASE-KEY.

- Array implementation optimal for dense graphs. $\leftarrow \Theta(n^2)$ edges
- Binary heap much faster for sparse graphs. $\leftarrow \Theta(n)$ edges
- 4-way heap worth the trouble in performance-critical situations.

priority queue	INSERT	DELETE-MIN	DECREASE-KEY	total
unordered array	$O(1)$	$O(n)$	$O(1)$	$O(n^2)$
binary heap	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(m \log n)$
d-way heap (Johnson 1975)	$O(d \log_d n)$	$O(d \log_d n)$	$O(\log_d n)$	$O(m \log_{m/n} n)$
Fibonacci heap (Fredman-Tarjan 1984)	$O(1)$	$O(\log n)^\dagger$	$O(1)^\dagger$	$O(m + n \log n)$
integer priority queue (Thorup 2004)	$O(1)$	$O(\log \log n)$	$O(1)$	$O(m + n \log \log n)$



How to solve the the single-source shortest paths problem in undirected graphs with positive edge weights?

- A.** Replace each undirected edge with two antiparallel edges of same weight. Run Dijkstra's algorithm in the resulting digraph.
- B.** Modify Dijkstra's algorithms so that when it processes node u , it consider all edges incident to u (instead of edges leaving u).
- C.** Either A or B.
- D.** Neither A nor B.



Theorem. [Thorup 1999] Can solve single-source shortest paths problem in undirected graphs with non-negative integer edge weights in $O(m)$ time.

Remark. Does not explore nodes in increasing order of distance from s .

Undirected Single Source Shortest Paths with Positive Integer Weights in Linear Time

Mikkel Thorup
AT&T Labs—Research

The single source shortest paths problem (SSSP) is one of the classic problems in algorithmic graph theory: given a positively weighted graph G with a source vertex s , find the shortest path from s to all other vertices in the graph.

Since 1959 all theoretical developments in SSSP for general directed and undirected graphs have been based on Dijkstra's algorithm, visiting the vertices in order of increasing distance from s . Thus, any implementation of Dijkstra's algorithm sorts the vertices according to their distances from s . However, we do not know how to sort in linear time.

Here, a deterministic linear time and linear space algorithm is presented for the undirected single source shortest paths problem with positive integer weights. The algorithm avoids the sorting bottleneck by building a hierarchical bucketing structure, identifying vertex pairs that may be visited in any order.

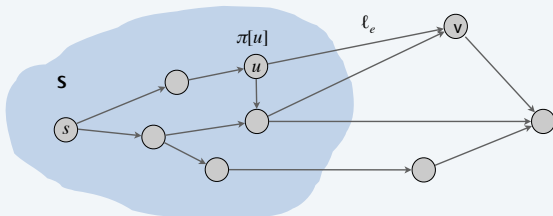


Extensions of Dijkstra's algorithm

Dijkstra's algorithm and proof extend to several related problems:

- Shortest paths in undirected graphs: $\pi[v] \leq \pi[u] + \ell(u, v)$.
- Maximum capacity paths: $\pi[v] \geq \min \{ \pi[u], c(u, v) \}$.
- Maximum reliability paths: $\pi[v] \geq \pi[u] \times \gamma(u, v)$.
- ...

Key algebraic structure. Closed semiring (min-plus, bottleneck, Viterbi, ...).



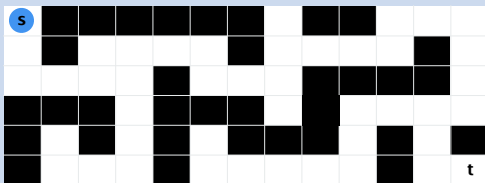
$$\begin{aligned}a + b &= b + a \\a + (b + c) &= (a + b) + c \\a + 0 &= a \\a \cdot (b \cdot c) &= (a \cdot b) \cdot c \\a \cdot 0 &= 0 \cdot a = 0 \\a \cdot 1 &= 1 \cdot a = a \\a \cdot (b + c) &= a \cdot b + a \cdot c \\(a + b) \cdot c &= a \cdot c + b \cdot c \\a^* &= 1 + a \cdot a^* = 1 + a^* \cdot a\end{aligned}$$

GOOGLE'S FOO.BAR CHALLENGE



You have maps of parts of the space station, each starting at a prison exit and ending at the door to an escape pod. The map is represented as a matrix of 0s and 1s, where 0s are passable space and 1s are impassable walls. The door out of the prison is at the top left $(0, 0)$ and the door into an escape pod is at the bottom right $(w-1, h-1)$.

Write a function that generates the length of a shortest path from the prison door to the escape pod, where you are allowed to **remove one wall** as part of your



GOOGLE'S FOO.BAR CHALLENGE

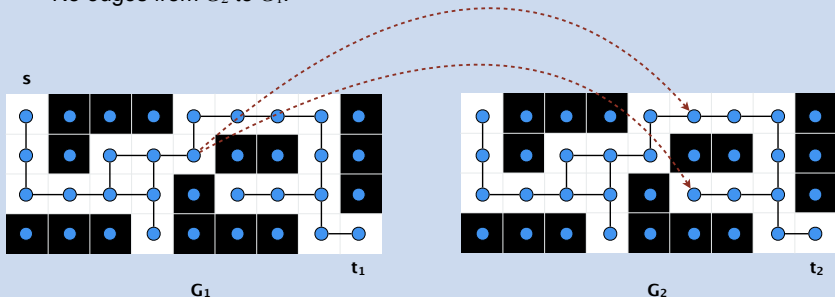


Idea 1. Model maze as a subgraph of grid graph.

- Node for each cell.
- Bidirectional edge between two adjacent open cells (length = 1).

Idea 2. Use “graph doubling” trick.

- An edge from G_1 to G_2 corresponds to removing a wall (length = 2).
- No edges from G_2 to G_1 .



Trang web có demo các thuật toán đồ thị

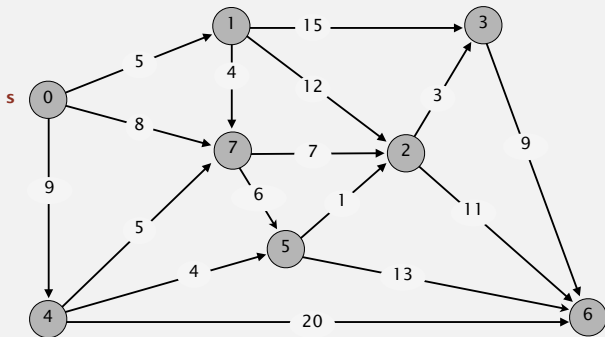
<https://www-m9.ma.tum.de/Algemeines/GraphAlgorithmenEn>

ĐƯỜNG ĐI NGẮN NHẤT

- ▶ *Đường đi ngắn nhất và thuật toán Dijkstra*
- ▶ **Đường đi ngắn nhất cho đồ thị DAG**
- ▶ *Thuật toán Bellman-Ford*
- ▶ *Thuật toán Floyd-Warshall*
- ▶ *Chu trình có trọng số âm*

Acyclic edge-weighted digraphs

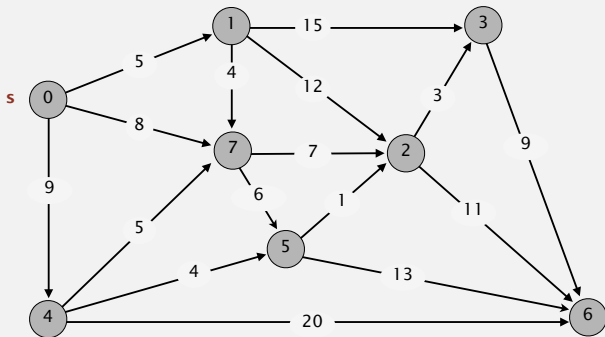
Q. Suppose that an edge-weighted digraph has no directed cycles. Is it easier to find shortest paths than in a general digraph?



A. Yes!

Acyclic shortest paths demo

- Consider vertices in topological order.
- Relax all edges pointing from that vertex.

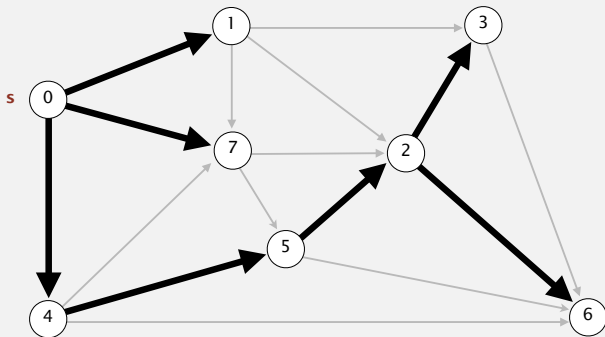


an edge-weighted DAG

0→1	5.0
0→4	9.0
0→7	8.0
1→2	12.0
1→3	15.0
1→7	4.0
2→3	3.0
2→6	11.0
3→6	9.0
4→5	4.0
4→6	20.0
4→7	5.0
5→2	1.0
5→6	13.0
7→5	6.0
7→2	7.0

Acyclic shortest paths demo

- Consider vertices in topological order.
- Relax all edges pointing from that vertex.



shortest-paths tree from vertex s

0 1 4 7 5 2 3 6

v	distTo[]	edgeTo[]
0	0.0	-
1	5.0	0→1
2	14.0	5→2
3	17.0	2→3
4	9.0	0→4
5	13.0	4→5
6	25.0	2→6
7	8.0	0→7

Shortest paths in edge-weighted DAGs

Proposition. Topological sort algorithm computes SPT in any edge-weighted DAG in time proportional to $E + V$.

edge weights
can be negative!

Pf.

- Each edge $e = v \rightarrow w$ is relaxed exactly once (when vertex v is relaxed), leaving $\text{distTo}[w] \leq \text{distTo}[v] + e.\text{weight}()$.
- Inequality holds until algorithm terminates because:
 - $\text{distTo}[w]$ cannot increase  $\text{distTo}[]$ values are monotone decreasing
 - $\text{distTo}[v]$ will not change  because of topological order, no edge pointing to v will be relaxed after v is relaxed
- Thus, upon termination, shortest-paths optimality conditions hold. ■

Shortest paths in edge-weighted DAGs

```
public class AcyclicSP
{
    private DirectedEdge[] edgeTo;
    private double[] distTo;

    public AcyclicSP(EdgeWeightedDigraph G, int s)
    {
        edgeTo = new DirectedEdge[G.V()];
        distTo = new double[G.V()];

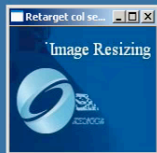
        for (int v = 0; v < G.V(); v++)
            distTo[v] = Double.POSITIVE_INFINITY;
        distTo[s] = 0.0;

        Topological topological = new Topological(G);
        for (int v : topological.order())
            for (DirectedEdge e : G.adj(v))
                relax(e);
    }
}
```

← topological order

Content-aware resizing

Seam carving. [Avidan and Shamir] Resize an image without distortion for display on cell phones and web browsers.



Shai Avidan
Mitsubishi Electric Research Lab
Ariel Shamir
The interdisciplinary Center & MERL

<http://www.youtube.com/watch?v=vIFCV2spKtg>

Content-aware resizing

Seam carving. [Avidan and Shamir] Resize an image without distortion for display on cell phones and web browsers.



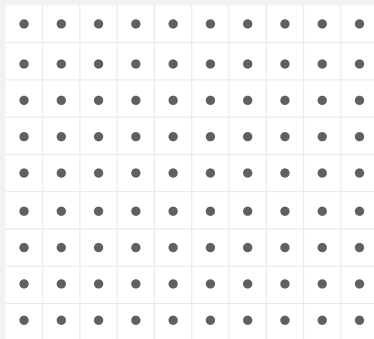
In the wild. Photoshop CS 5, Imagemagick, GIMP, ...



Content-aware resizing

To find vertical seam:

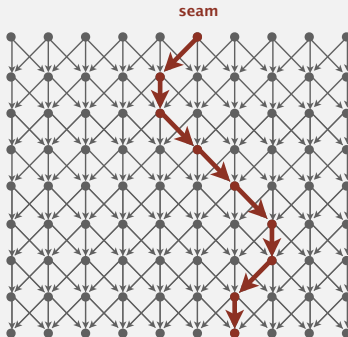
- Grid DAG: vertex = pixel; edge = from pixel to 3 downward neighbors.
- Weight of pixel = energy function of 8 neighboring pixels.
- Seam = shortest path (sum of vertex weights) from top to bottom.



Content-aware resizing

To find vertical seam:

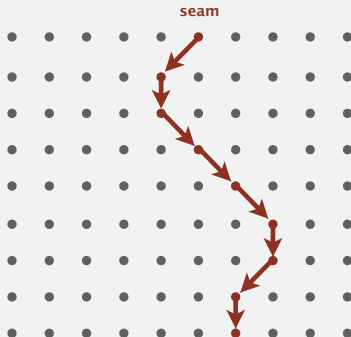
- Grid DAG: vertex = pixel; edge = from pixel to 3 downward neighbors.
- Weight of pixel = energy function of 8 neighboring pixels.
- Seam = shortest path (sum of vertex weights) from top to bottom.



Content-aware resizing

To remove vertical seam:

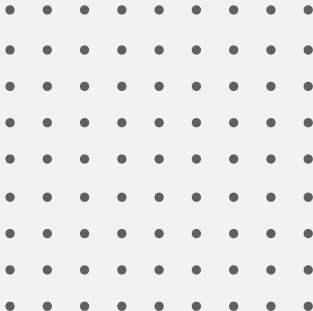
- Delete pixels on seam (one in each row).



Content-aware resizing

To remove vertical seam:

- Delete pixels on seam (one in each row).



Longest paths in edge-weighted DAGs

Formulate as a shortest paths problem in edge-weighted DAGs.

- Negate all weights.
- Find shortest paths.
- Negate weights in result.

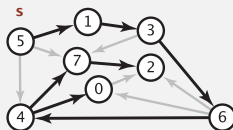
equivalent: reverse sense of equality in `relax()`

longest paths input

5→4 0.35
4→7 0.37
5→7 0.28
5→1 0.32
4→0 0.38
0→2 0.26
3→7 0.39
1→3 0.29
7→2 0.34
6→2 0.40
3→6 0.52
6→0 0.58
6→4 0.93

shortest paths input

5→4 -0.35
4→7 -0.37
5→7 -0.28
5→1 -0.32
4→0 -0.38
0→2 -0.26
3→7 -0.39
1→3 -0.29
7→2 -0.34
6→2 -0.40
3→6 -0.52
6→0 -0.58
6→4 -0.93



Key point. Topological sort algorithm works even with negative weights.

Longest paths in edge-weighted DAGs: application

Parallel job scheduling. Given a set of jobs with durations and precedence constraints, schedule the jobs (by finding a start time for each) so as to achieve the minimum completion time, while respecting the constraints.

<i>job</i>	<i>duration</i>	<i>must complete before</i>
------------	-----------------	-----------------------------

0	41.0	1 7 9
1	51.0	2
2	50.0	
3	36.0	
4	38.0	
5	45.0	
6	21.0	3 8
7	32.0	3 8
8	32.0	2
9	29.0	4 6



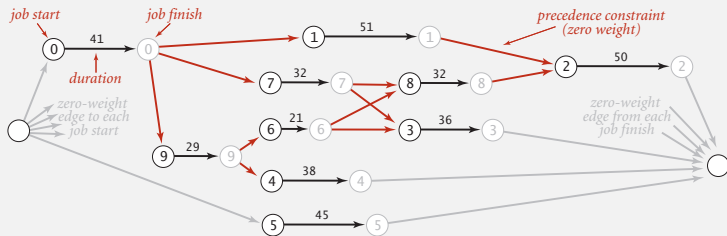
Parallel job scheduling solution

Critical path method

CPM. To solve a parallel job-scheduling problem, create edge-weighted DAG:

- Source and sink vertices.
- Two vertices (begin and end) for each job.
- Three edges for each job.
 - begin to end (weighted by duration)
 - source to begin (0 weight)
 - end to sink (0 weight)
- One edge for each precedence constraint (0 weight).

job	duration	must complete before		
0	41.0	1	7	9
1	51.0	2		
2	50.0			
3	36.0			
4	38.0			
5	45.0			
6	21.0	3	8	
7	32.0	3	8	
8	32.0	2		
9	29.0	4	6	

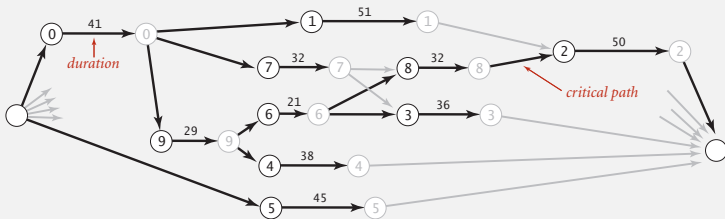


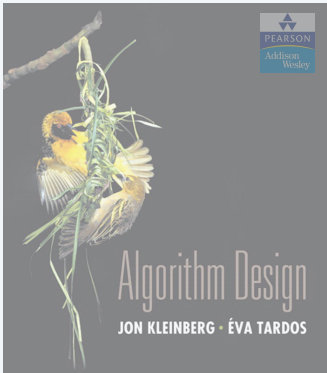
Critical path method

CPM. Use **longest path** from the source to schedule each job.



Parallel job scheduling solution





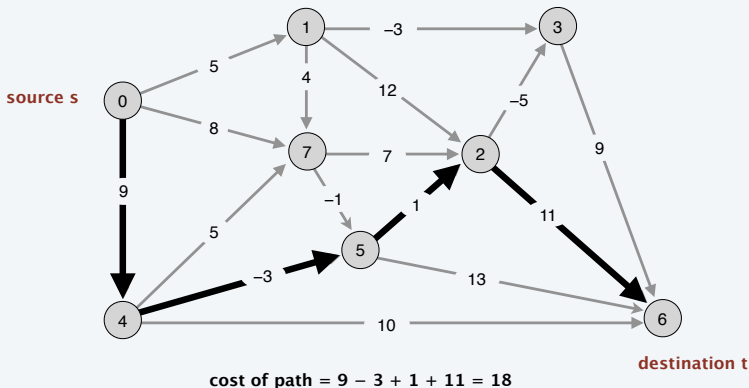
SECTION 6.8

ĐƯỜNG ĐI NGẮN NHẤT

- ▶ *Đường đi ngắn nhất và thuật toán Dijkstra*
- ▶ *Đường đi ngắn nhất cho đồ thị DAG*
- ▶ **Thuật toán Bellman-Ford**
- ▶ *Thuật toán Floyd-Warshall*
- ▶ *Chu trình có trọng số âm*

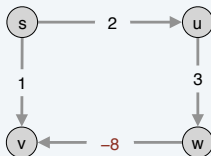
Shortest paths

Shortest-path problem. Given a digraph $G = (V, E)$, with arbitrary edge weights or costs c_{vw} , find cheapest path from node s to node t .

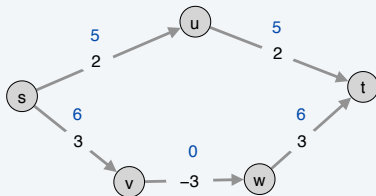


Shortest paths: failed attempts

Dijkstra. May not produce shortest paths when edge weights are negative.

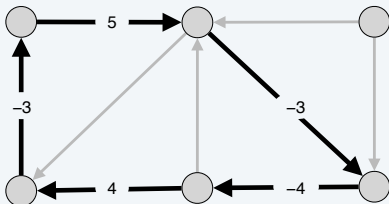


Reweighting. Adding a constant to every edge weight does not necessarily make Dijkstra's algorithm produce shortest paths.



Negative cycles

Def. A **negative cycle** is a directed cycle such that the sum of its edge weights is negative.

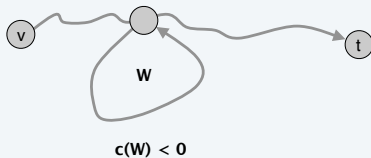


a negative cycle W : $c(W) = \sum_{e \in W} c_e < 0$

Shortest paths and negative cycles

Lemma 1. If some path from v to t contains a negative cycle, then there does not exist a cheapest path from v to t .

Pf. If there exists such a cycle W , then can build a $v \rightarrow t$ path of arbitrarily negative weight by detouring around cycle as many times as desired. ■

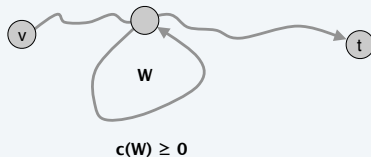


Shortest paths and negative cycles

Lemma 2. If G has no negative cycles, then there exists a cheapest path from v to t that is simple (and has $\leq n - 1$ edges).

Pf.

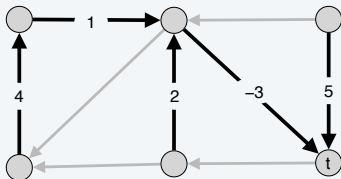
- Consider a cheapest $v \rightarrow t$ path P that uses the fewest edges.
- If P contains a cycle W , can remove portion of P corresponding to W without increasing the cost. ▀



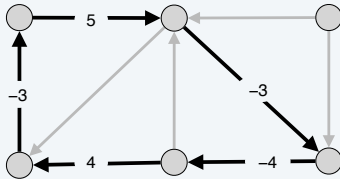
Shortest-paths and negative-cycle problems

Single-destination shortest-paths problem. Given a digraph $G = (V, E)$ with edge weights c_{vw} and no negative cycles and a distinguished node t , find cheapest $v \rightarrow t$ path for each node v .

Negative-cycle problem. Given a digraph $G = (V, E)$ with edge weights c_{vw} , find a negative cycle (if one exists).



shortest-paths tree



negative cycle

Shortest paths: dynamic programming

Def. $OPT(i, v)$ = cost of shortest $v \rightarrow t$ path that uses $\leq i$ edges.

- Case 1: Cheapest $v \rightarrow t$ path uses $\leq i - 1$ edges.
 - $OPT(i, v) = OPT(i - 1, v)$
- Case 2: Cheapest $v \rightarrow t$ path uses exactly i edges.
 - if (v, w) is first edge, then OPT uses (v, w) , and then selects best $w \rightarrow t$ path using $\leq i - 1$ edges

 optimal substructure property
(proof via exchange argument)

$$OPT(i, v) = \begin{cases} \infty & \text{if } i = 0 \\ \min \left\{ OPT(i-1, v), \min_{(v, w) \in E} \{ OPT(i-1, w) + c_{vw} \} \right\} & \text{otherwise} \end{cases}$$

Observation. If no negative cycles, $OPT(n - 1, v)$ = cost of cheapest $v \rightarrow t$ path.

Pf. By Lemma 2, cheapest $v \rightarrow t$ path is simple. ■

Shortest paths: implementation

SHORTEST-PATHS (V, E, c, t)

FOREACH node $v \in V$

$M[0, v] \leftarrow \infty.$

$M[0, t] \leftarrow 0.$

FOR $i = 1$ TO $n - 1$

FOREACH node $v \in V$

$M[i, v] \leftarrow M[i - 1, v].$

FOREACH edge $(v, w) \in E$

$M[i, v] \leftarrow \min \{ M[i, v], M[i - 1, w] + c_{vw} \}.$

Shortest paths: implementation

Theorem 1. Given a digraph $G = (V, E)$ with no negative cycles, the dynamic programming algorithm computes the cost of a cheapest $v \rightarrow t$ path for each node v in $\Theta(mn)$ time and $\Theta(n^2)$ space.

Pf.

- Table requires $\Theta(n^2)$ space.
- Each iteration i takes $\Theta(m)$ time since we examine each edge once. ■

Finding the shortest paths.

- Approach 1: Maintain a *successor*(i, v) that points to next node on cheapest $v \rightarrow t$ path using at most i edges.
- Approach 2: Compute optimal costs $M[i, v]$ and consider only edges with $M[i, v] = M[i - 1, w] + c_{vw}$.

Shortest paths: practical improvements

Space optimization. Maintain two 1d arrays (instead of 2d array).

- $d(v)$ = cost of a cheapest $v \rightarrow t$ path that we have found so far.
- $successor(v)$ = next node on a $v \rightarrow t$ path.

Performance optimization. If $d(w)$ was not updated in iteration $i - 1$, then no reason to consider edges entering w in iteration i .

Bellman–Ford: efficient implementation

BELLMAN–FORD (V, E, c, t)

FOREACH node $v \in V$

$d(v) \leftarrow \infty$.

$successor(v) \leftarrow null$.

$d(t) \leftarrow 0$.

FOR $i = 1$ TO $n - 1$

FOREACH node $w \in V$

IF ($d(w)$ was updated in previous iteration)

FOREACH edge $(v, w) \in E$

IF ($d(v) > d(w) + c_{vw}$)

$d(v) \leftarrow d(w) + c_{vw}$.

$successor(v) \leftarrow w$.

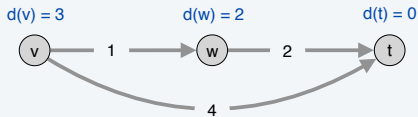
IF no $d(w)$ value changed in iteration i , STOP.

1 pass

Bellman-Ford: analysis

Claim. After the i^{th} pass of Bellman-Ford, $d(v)$ equals the cost of a cheapest $v \rightarrow t$ path using at most i edges.

Counterexample. Claim is false!



if node w considered before node v ,
then $d(v) = 3$ after 1 pass

Bellman–Ford: analysis

Lemma 3. Throughout Bellman–Ford algorithm, $d(v)$ is the cost of some $v \rightarrow t$ path; after the i^{th} pass, $d(v)$ is no larger than the cost of a cheapest $v \rightarrow t$ path using $\leq i$ edges.

Pf. [by induction on i]

- Assume true after i^{th} pass.
- Let P be any $v \rightarrow t$ path with $i + 1$ edges.
- Let (v, w) be first edge on path and let P' be subpath from w to t .
- By inductive hypothesis, $d(w) \leq c(P')$ since P' is a $w \rightarrow t$ path with i edges.
- After considering v in pass $i + 1$:

$$\begin{aligned}d(v) &\leq c_{vw} + d(w) \\&\leq c_{vw} + c(P') \\&= c(P) \quad \blacksquare\end{aligned}$$

Theorem 2. Given a digraph with no negative cycles, Bellman–Ford computes the costs of the cheapest $v \rightarrow t$ paths in $O(mn)$ time and $\Theta(n)$ extra space.

Pf. Lemmas 2 + 3. $\blacksquare$


can be substantially
faster in practice

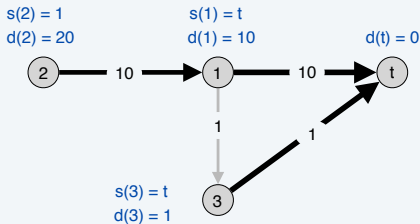
Bellman-Ford: analysis

Claim. Throughout the Bellman-Ford algorithm, following ~~$\text{successor}(v)$~~ pointers gives a directed path from v to t of cost $d(v)$.

Counterexample. Claim is false!

- Cost of successor $v \rightarrow t$ path may have strictly lower cost than $d(v)$.

consider nodes in order: $t, 1, 2, 3$



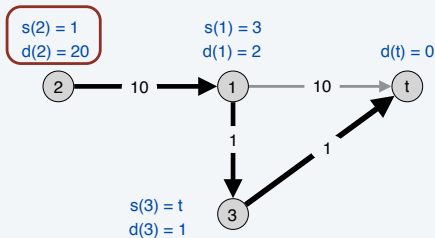
Bellman-Ford: analysis

Claim. Throughout the Bellman-Ford algorithm, following ~~successor(v)~~ pointers gives a directed path from v to t of cost $d(v)$.

Counterexample. Claim is false!

- Cost of successor $v \rightarrow t$ path may have strictly lower cost than $d(v)$.

consider nodes in order: $t, 1, 2, 3$



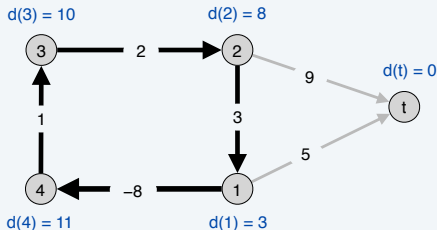
Bellman–Ford: analysis

Claim. Throughout the Bellman-Ford algorithm, following $\text{successor}(v)$ pointers gives a directed path from v to t of cost $d(v)$.

Counterexample. Claim is false!

- Cost of successor $v \rightarrow t$ path may have strictly lower cost than $d(v)$.
- Successor graph may have cycles.

consider nodes in order: t, 1, 2, 3, 4



Bellman–Ford: finding the shortest paths

Lemma 4. If the successor graph contains a directed cycle W , then W is a negative cycle.

Pf.

- If $\text{successor}(v) = w$, we must have $d(v) \geq d(w) + c_{vw}$.
(LHS and RHS are equal when $\text{successor}(v)$ is set; $d(w)$ can only decrease; $d(v)$ decreases only when $\text{successor}(v)$ is reset)
- Let $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k$ be the nodes along the cycle W .
- Assume that (v_k, v_1) is the last edge added to the successor graph.

- Just prior to that:
$$\begin{array}{rcl} d(v_1) & \geq & d(v_2) + c(v_1, v_2) \\ d(v_2) & \geq & d(v_3) + c(v_2, v_3) \\ \vdots & & \vdots \\ d(v_{k-1}) & \geq & d(v_k) + c(v_{k-1}, v_k) \\ d(v_k) & > & d(v_1) + c(v_k, v_1) \end{array}$$

← holds with strict inequality
since we are updating $d(v_k)$

- Adding inequalities yields $c(v_1, v_2) + c(v_2, v_3) + \dots + c(v_{k-1}, v_k) + c(v_k, v_1) < 0$. ▀

W is a negative cycle

Bellman–Ford: finding the shortest paths

Theorem 3. Given a digraph with no negative cycles and a node t , Bellman–Ford finds cheapest $v \rightarrow t$ paths for each node v in $O(mn)$ time and $\Theta(n)$ extra space.

Pf.

- The successor graph cannot have a negative cycle. [Lemma 4]
- Thus, following the successor pointers from v yields a directed path to t .
- Let $v = v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k = t$ be the nodes along this path P .
- Upon termination, if $\text{successor}(v) = w$, we must have $d(v) = d(w) + c_{vw}$.

(LHS and RHS are equal when $\text{successor}(v)$ is set; $d(\cdot)$ did not change)

$$\begin{aligned} \bullet \text{ Thus, } \quad d(v_1) &= d(v_2) + c(v_1, v_2) \\ \bullet \quad \quad d(v_2) &= d(v_3) + c(v_2, v_3) \\ &\vdots \\ d(v_{k-1}) &= d(v_k) + c(v_{k-1}, v_k) \end{aligned}$$

↑
since algorithm
terminated

Adding equations yields $d(s) = d(t) + c(v_1, v_2) + c(v_2, v_3) + \dots + c(v_{k-1}, v_k)$. ■

↑ min cost of any $v \rightarrow t$ path (Theorem 2) 0 $\xrightarrow{\text{cost of path } P}$

Single source shortest-paths implementation: cost summary

algorithm	restriction	typical case	worst case	extra space
topological sort	no directed cycles	$E + V$	$E + V$	V
Dijkstra (binary heap)	no negative weights	$E \log V$	$E \log V$	V
Bellman-Ford	no negative cycles	$E V$	$E V$	V
Bellman-Ford (queue-based)		$E + V$	$E V$	V

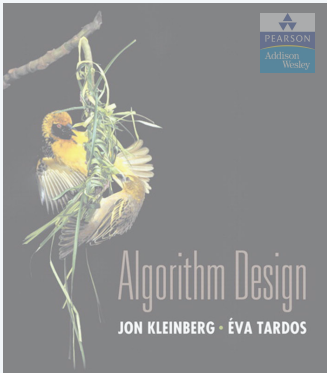
Remark 1. Directed cycles make the problem harder.

Remark 2. Negative weights make the problem harder.

Remark 3. Negative cycles makes the problem intractable.

ĐƯỜNG ĐI NGẮN NHẤT

- ▶ Đường đi ngắn nhất và thuật toán Dijkstra
- ▶ Đường đi ngắn nhất cho đồ thị DAG
- ▶ Thuật toán Bellman-Ford
- ▶ Thuật toán Floyd-Warshall
- ▶ Chu trình có trọng số âm



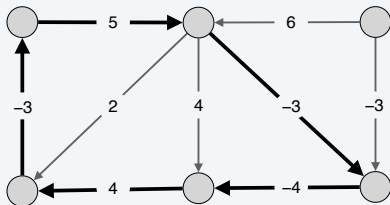
SECTION 6.10

ĐƯỜNG ĐI NGẮN NHẤT

- ▶ *Đường đi ngắn nhất và thuật toán Dijkstra*
- ▶ *Đường đi ngắn nhất cho đồ thị DAG*
- ▶ *Thuật toán Bellman-Ford*
- ▶ *Thuật toán Floyd-Warshall*
- ▶ *Chu trình có trọng số âm*

Detecting negative cycles

Negative cycle detection problem. Given a digraph $G = (V, E)$, with edge weights c_{vw} , find a negative cycle (if one exists).

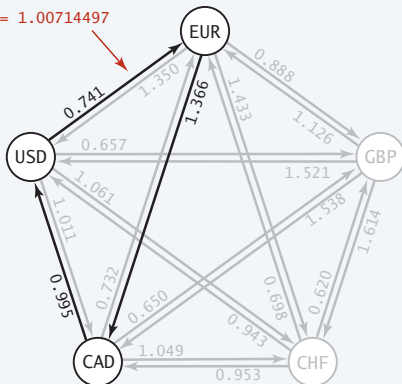


Detecting negative cycles: application

Currency conversion. Given n currencies and exchange rates between pairs of currencies, is there an arbitrage opportunity?

Remark. Fastest algorithm very valuable!

$$0.741 * 1.366 * .995 = 1.00714497$$



Detecting negative cycles

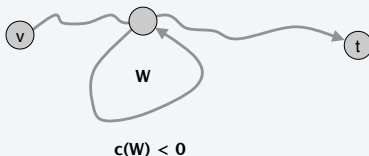
Lemma 5. If $OPT(n, v) = OPT(n - 1, v)$ for all v , then no negative cycle can reach t .

Pf. Bellman–Ford algorithm. ▀

Lemma 6. If $OPT(n, v) < OPT(n - 1, v)$ for some node v , then (any) cheapest path from v to t contains a cycle W . Moreover W is a negative cycle.

Pf. [by contradiction]

- Since $OPT(n, v) < OPT(n - 1, v)$, we know that shortest $v \rightarrow t$ path P has exactly n edges.
- By pigeonhole principle, P must contain a directed cycle W .
- Deleting W yields a $v \rightarrow t$ path with $< n$ edges $\Rightarrow W$ has negative cost. ▀

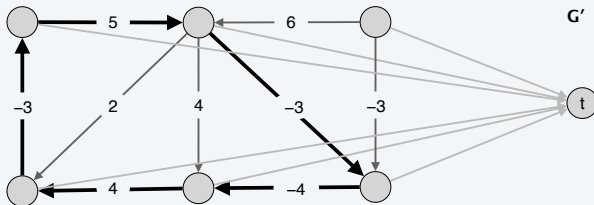


Detecting negative cycles

Theorem 4. Can find a negative cycle in $\Theta(mn)$ time and $\Theta(n^2)$ space.

Pf.

- Add new node t and connect all nodes to t with 0-cost edge.
- G has a negative cycle iff G' has a negative cycle than can reach t .
- If $OPT(n, v) = OPT(n - 1, v)$ for all nodes v , then no negative cycles.
- If not, then extract directed cycle from path from v to t .
(cycle cannot contain t since no edges leave t) ■



Detecting negative cycles

Theorem 5. Can find a negative cycle in $O(mn)$ time and $O(n)$ extra space.

Pf.

- Run Bellman–Ford for n passes (instead of $n - 1$) on modified digraph.
- If no $d(v)$ values updated in pass n , then no negative cycles.
- Otherwise, suppose $d(s)$ updated in pass n .
- Define $pass(v)$ = last pass in which $d(v)$ was updated.
- Observe $pass(s) = n$ and $pass(successor(v)) \geq pass(v) - 1$ for each v .
- Following successor pointers, we must eventually repeat a node.
- Lemma 4 $\Rightarrow$ this cycle is a negative cycle. ▪

Remark. See p. 304 for improved version and early termination rule.

(Tarjan's subtree disassembly trick)

Bình luận cuối cùng

Tốc độ tính toán